

Automated TOS & Exam Generator

Multi-PDF Table of Specification and exam builder for teachers.

Laravel 12 + PHP 8.2 + Google Gemini + local Bloom classifier.

Audience: teachers, school admins, and developers evaluating, extending, or retraining the system. This document covers what the project is, exactly how lesson files are processed, and how the built-in machine-learning model is trained, replaced, and evaluated.

1. What this project is

A web application that turns a teacher's lesson PDFs and learning outcomes into a balanced Table of Specification (TOS) and a finished exam. The teacher uploads one PDF per lesson (1-15 lessons), states how much class time each lesson received, and pastes in each lesson's learning outcomes. The system classifies every outcome against Bloom's Taxonomy, apportions exam items fairly across lessons and cognitive levels, and then generates the actual questions with AI -- each question traceable to its source lesson.

- **Manual TOS tables are slow and error-prone.** Weighting lessons and balancing Bloom's levels by hand takes hours and rarely sums exactly.
- **Single-prompt AI exam generation is uneven.** Stuffing many lessons into one AI call dilutes context and skips lessons. This system gives each lesson its own scoped AI call.
- **No traceability.** Every generated question is tagged with its lesson and Bloom's level, so coverage can be verified.

2. Lesson processing, step by step (Part 1: upload + TOS)

Upload and exam generation are two separate requests. Phase 1 begins when the teacher submits the form to POST /tos:

- **Step 1 - Validate.** The system checks the course name, total exam items (5-200), and a lessons array (1-15 entries). Each lesson must have a title, a weight from 0.1 to 100, objectives text, and a PDF file (verified PDF format, max 20 MB).
- **Step 2 - Allocate items first.** Before anything is stored, TosBuilderService splits the total item count across lessons by teacher weight (class hours/emphasis, not page count), then splits each lesson's quota across the six Bloom's levels using the standard 20/20/20/15/15/10 balance. Largest-remainder arithmetic guarantees the counts always sum exactly.
- **Step 3 - Store and classify in one database transaction.** For each lesson the PDF is saved to storage/app/private/lesson_pdfs/ (only

the relative path is kept in the database), a Lesson record is created with its quota, and the objectives text is split on line breaks, trimmed, and cleaned. Every single objective is then classified by the local Bloom model and saved with its text, level, confidence score, and full probability distribution.

3. Lesson processing, step by step (Part 2: exam generation)

Phase 2 begins when the teacher clicks Generate Exam (POST /tos/{tos}/generate-exam). The request is allowed up to 300 seconds because multi-lesson AI generation is slow by design:

- **Step 4 - Extract text per PDF.** Each stored PDF is resolved to its absolute path and parsed with the `smalot/pdfparser` library; whitespace noise is collapsed. The text is truncated to 10,000 characters per lesson to keep AI cost and latency predictable.
- **Step 5 - Skip unreadables.** Lessons with empty extracted text -- typical for scanned or image-only PDFs, since no OCR is built in -- are skipped and named in the status message. If no lesson yields any text, the request fails with a clear error.
- **Step 6 - Generate concurrently.** The `ExamGeneratorService` fires one Google Gemini call per lesson in parallel, so ten lessons finish in roughly the time of one or two sequential calls. Rate-limit responses (429/503) are retried one at a time with doubling backoff (max 3 attempts); other failures affect only their own lesson. Remembering/Understanding levels produce Multiple Choice + Enumeration questions; higher levels produce Modified True-or-False + Enumeration. Any missing level/type combination is automatically requested again in a follow-up prompt (up to 2 passes).
- **Step 7 - Persist per lesson.** Each lesson's questions are saved under that lesson. If some lessons succeed and others fail, the response returns partial-success status with a per-lesson error list. Any failed lesson can later be retried alone via POST /tos/{tos}/lessons/{lesson}/generate-exam without touching the rest.

4. The algorithm: trained in advance, not live

Bloom classification uses a multinomial logistic-regression model written in pure PHP -- no external ML library, so it runs on any basic PHP/XAMPP setup. It is trained offline, before any web request, with the command ``php artisan bloom:train`` (defaults: 300 epochs, learning rate 0.5). Training writes a weights file to `storage/app/ml/bloom_model.json`; the web app only loads those weights to make predictions. No learning happens during requests. The model file is generated, not committed to the repository, so every fresh installation must run the training command once or classification will fail with an explicit error.

At prediction time each objective is tokenized (lowercased, punctuation stripped, words of 3+ letters kept), converted to a bag-of-words vector, scored against the trained weights, and passed through softmax to pick one of Remembering, Understanding, Applying, Analyzing, Evaluating, or Creating, with a confidence percentage.

5. Training a different model and testing competency

Yes, a different model can be trained: add or replace labeled rows, adjust hyperparameters, or swap in another algorithm entirely (naive Bayes, an exported scikit-learn model, even an LLM-based classifier) -- anything that returns the same `classify()` result shape plugs in cleanly. One limitation: the system supports a single active model at one fixed file path, with no versioning or A/B switch, so retraining overwrites the previous model.

Competency testing must be built -- none exists today (the test suite holds only framework stubs, and training reports sample counts but no accuracy). The sound approach is a holdout split: train on about 80% of the labeled data and report accuracy plus per-class precision/recall on the untouched 20%, or use k-fold cross-validation given the small dataset. A minimum-accuracy automated test then guards against regressions. Never evaluate on training data: with only 60 samples, training accuracy looks flattering and means little.

6. Training data: what it is, and where more comes from

The current dataset is 60 hand-written generic sentences -- 10 per Bloom level -- stored in `app/Services/Bloom/BloomTrainingData.php`, nearly all of the form "Students will [verb] ..." (define, list, explain, solve, analyze, evaluate, design...). The file's own notes ask for real DepEd objectives, acknowledging this is placeholder data.

- **From the net (partial help).** Bloom's verb lists are widely published by university teaching centers and improve vocabulary coverage. But they mismatch this model, which learns from full objective sentences sharing a common boilerplate -- a bare verb list teaches neither sentence patterns nor consistent labeling.
- **Your own data (what actually works).** Real objectives in the phrasing teachers actually paste in, labeled by level with expert review: hundreds of them,

reasonably balanced across all six levels, plus a held-out test set never used in training. The practical source is TOS documents and lesson plans you already own -- collect, label, append, retrain, and compare against the previous model with the holdout evaluation above.

7. Key features

- **Per-lesson pipeline.** Independent classification, quota, and generation per lesson PDF.
- **Exact allocation.** Two-level apportionment -- totals always reconcile.
- **Concurrent AI generation.** Parallel per-lesson calls finish in roughly the time of 1-2 sequential calls.
- **Three question types.** Multiple choice, modified true-or-false (with correction), and enumeration.
- **Single-page interface.** One page; all actions happen via fetch with no reloads, plus a history sidebar of past TOS records.
- **Local Bloom classifier.** Pure-PHP model trained with one artisan command -- no external ML service.

8. Technology stack

- **Backend:** Laravel 12, PHP 8.2, SQLite (default), Eloquent ORM.
- **AI:** Google Gemini (gemini-2.5-flash-lite) via concurrent HTTP pool with retry/backoff.
- **ML:** Logistic-regression Bloom classifier in pure PHP; PDF parsing via smalot/pdfparser.
- **Frontend:** Blade templates, Tailwind CSS v4, Vite; vanilla JS fetch.
- **Tooling:** PHPUnit (SQLite in-memory), Laravel Pint, Pail, Concurrently dev runner.

9. Typical workflow

- **1.** Open /tos/create; enter the course name and total exam items.
- **2.** For each lesson: title, weight (class hours), learning outcomes (one per line), and the lesson PDF.
- **3.** Submit, review the TOS balance, then click Generate Exam.
- **4.** Review the exam grouped by lesson; retry any failed lesson individually.

10. Current limitations

- **Scanned PDFs need OCR.** Image-only PDFs yield no text and are skipped; OCR is not built in.
- **No per-lesson Bloom override.** All lessons share the same global Bloom percentage balance.
- **Single active ML model.** No model versioning, A/B testing, or accuracy harness yet.
- **Fixed AI provider.** Exam generation is hardcoded to one Gemini model and one server-side API key -- see Task 1 below.

- **No auth yet.** History is unscoped; user accounts are a future addition.

11. Planned tasks

Task 1 - Implement AI Configuration Window. Today the AI provider is fixed: one hardcoded Gemini model plus a single server-side GEMINI_API_KEY in .env, so every teacher shares the same model and quota. The goal is a settings window where each user chooses their own model by providing three things:

- **API key.** The user's own key for their chosen provider, stored securely (encrypted at rest), never displayed back in full.
- **Provider URL.** The API endpoint/base URL of the provider (e.g. Google Gemini, OpenAI-compatible endpoints, or a self-hosted gateway), so generation is not locked to one vendor.
- **Model name.** The exact model to call (e.g. gemini-2.5-flash-lite or any compatible model the provider offers).

Acceptance: exam generation reads the AI configuration from the user's saved settings instead of the hardcoded constants; a test-connection action validates the key/URL/model before saving; a missing or invalid configuration produces a clear in-app error rather than a silent failure; existing behavior (server default) keeps working when the user has configured nothing.